

Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias

Analizador Sintáctico

Triton GPU Kernel

TC3002B — Computer Science Advanced Applications Development
Módulo #3: Compiladores — Fase de Análisis Sintáctico

Equipo: 11

Cristóbal Camarena Hernández — A01642653

Josué Galindo Gutiérrez — A01637983

Gandhi Emmanuel Valdez Huerta — A01625738

Diego Alberto Estrada López — A01638657

Profesor: Dr. Salvador Hinojosa

Fecha de entrega: Junio 2026

Guadalajara, Jalisco, México

Resumen

Resultados: el analizador sintáctico `triton_parser` obtiene **12/12 PASS** en la suite unitaria y **68/70 PARSE_OK (97.1 %)** en el benchmark TRITONHEAL. Sobre kernels **válidos**, la tasa de aceptación es **100 % (68/68)**; los 2 rechazos son mutaciones inválidas (`k_0040`, `k_0058`) confirmadas con `ast.parse`. Este reporte documenta la CFG, el diseño yacc/Flex, las tablas de símbolos, los mensajes de error y la evidencia experimental con gráfica y tablas de resultados.

Índice

1. Introducción	1
1.1. Resumen	1
1.2. Notación	1
1.3. Justificación de herramientas	1
1.4. Trazabilidad a la fase léxica	1
2. Análisis	1
2.1. Entregables del reto (Sección IV, documento de evaluación)	1
2.2. Requerimientos funcionales	2
2.3. CFG (resumen)	2
2.4. Pasos para obtener la gramática	2
2.5. Simplificaciones justificadas	3
2.6. Modificaciones al analizador léxico	3
2.7. Patrones léxicos: <code>BAD_ID</code>	3
2.8. Trazabilidad token léxico $\rightarrow$ yacc	4
3. Diseño	4
3.1. Arquitectura del sistema	4
3.2. Integración <code>lex</code> $\leftrightarrow$ yacc	4
3.3. Diseño de la especificación yacc	4
3.4. Tabla de símbolos del parser (diseño)	5
3.5. Mensajes de error (diseño)	5
3.6. Trazabilidad diseño $\rightarrow$ análisis	5
4. Implementación	5
4.1. Estructura del proyecto	5
4.2. Compilación y ejecución	5
4.3. Fragmento: detección <code>BAD_ID</code> (lexer)	5
4.4. Fragmento: integración de indentación (lexer)	6
4.5. Fragmento: reglas principales (yacc)	7
4.6. Fragmento: tabla de símbolos	8
4.7. Fragmento: manejo de errores	9
4.8. Ejemplo de salida del parser (entregable 5)	10
5. Verificación y Validación	10
5.1. Resultados experimentales	10
5.2. Resumen cuantitativo	11
5.3. Interpretación de los resultados	11
5.4. Gráfica de resultados (Figura 2)	11
5.5. Detalle de los 2 rechazos (comportamiento esperado)	13
5.6. Modelo de pruebas	13
5.7. Suite unitaria (<code>make test</code>)	13

5.8. Evidencia: salida exitosa (<code>ejemplo_1.tri</code>)	13
5.9. Evidencia: error sintáctico (<code>def_sin_dos_puntos.tri</code>)	14
5.10. Evidencia: error léxico <code>BAD_ID</code> (I6)	14
5.11. Reproducibilidad	14
5.11.1. Prerrequisitos	14
5.11.2. Pasos desde cero (PC nueva)	15
5.11.3. Comando único (regeneración completa)	15
5.11.4. Salidas verificables	15
5.11.5. Consulta sin recompilar	15
5.11.6. Notas por sistema operativo	15
6. Plan de Trabajo	16
6.1. Distribución de responsabilidades	16
A. Gramática libre de contexto completa	16
A.1. Convenciones y notación	16
A.2. Símbolo inicial y conteo	17
A.3. Terminales del analizador	17
A.3.1. Terminales que aparecen en producciones	17
A.3.2. Terminales declarados y no usados en reglas	17
A.4. Precedencia y asociatividad (<code>yacc</code>)	18
A.5. Inventario de no terminales (33)	18
A.6. Notas semánticas ligadas a la CFG (fuera de producciones)	19
A.7. Desglose detallado: no terminal <code>expr</code> (16 alternativas)	19
A.8. Desglose detallado: <code>subscript</code> (5 alternativas)	20
A.9. Desglose detallado: <code>block_stmt</code> (3 alternativas)	20
A.10. Listado completo de producciones (P-001 a P-091)	20
A.11. Implementación <code>yacc</code> (referencia cruzada)	23
A.12. Trazabilidad y mantenimiento	27
B. Referencias	27

Índice de figuras

1. Pipeline del analizador sintáctico.	4
2. Resultados del analizador sintáctico. Panel superior: indicadores clave. Centro izquierda: desglose (68 válidos aceptados, 2 mutaciones rechazadas, 0 falsos rechazos). Centro derecha: 97.1 % <code>PARSE_OK</code> global. Inferior: mapa de los 70 kernels (<code>k_0000–k_0069</code>); verde = éxito, naranja = mutación inválida (<code>k_0040</code> , <code>k_0058</code>).	12

Índice de cuadros

1. Simplificaciones de la CFG.	3
2. Caracteres invalidos en identificadores y patron Flex.	3
3. Correspondencia Token ID (reporte léxico) y <code>yacc</code>	4
4. Esquema de entradas en <code>syntab.c</code>	5
5. Cuadro resumen de resultados (corridas locales reproducibles con <code>make test</code> y <code>make benchmark</code>).	11
6. Kernels rechazados: mutaciones inválidas del benchmark.	13
7. Casos de prueba del equipo (11 ejecuciones).	13

8.	Software necesario para reproducir compilación, pruebas, gráfica e informe. . . .	14
9.	Cronograma de la fase sintáctica (Equipo 11).	16
10.	Resumen formal de la gramática implementada.	17
11.	Terminales usados en al menos una producción de la CFG.	17
12.	Directivas <code>%left</code> / <code>%nonassoc</code> en <code>parser.y</code> (L64–70).	18
13.	Catálogo de no terminales: número de alternativas e intervalo en <code>parser.y</code>	18
14.	Acciones en C asociadas a reglas seleccionadas (no son parte de la CFG).	19

1. Introducción

1.1. Resumen

Este documento presenta el análisis, diseño, implementación y verificación del **analizador sintáctico** para kernels **Triton GPU** escritos en archivos `.tri`. La fase sintáctica es la segunda etapa del compilador del reto académico: consume la secuencia de tokens producida por el analizador léxico (Flex) del Equipo 11, construye una estructura conforme a una gramática libre de contexto (CFG) implementada con **yacc/Bison**, actualiza tablas de símbolos y reporta errores sintácticos.

El entregable incluye: (i) gramática documentada (Apéndice A, CFG completa); (ii) tablas de símbolos; (iii) implementación `triton_parser`; (iv) mensajes de error; (v) **resultados experimentales** con énfasis en la Sección 5.1: **100 %** suite unitaria, **97.1 %** benchmark (68/70), **100 %** en kernels válidos, y gráfica detallada de los 70 kernels.

1.2. Notación

- **CFG**: gramática libre de contexto en notación BNF; producciones $A \rightarrow \alpha$.
- **Token ID**: identificador numérico del Cuadro 1 del reporte léxico (100–999).
- **INDENT/DEDENT** (910/911): tokens sintéticos derivados de `WS_COUNT` (900) para modelar indentación Python.
- **PARSE_OK**: análisis sintáctico exitoso con impresión de tabla de símbolos del parser.

1.3. Justificación de herramientas

Se utiliza **C** con **Flex** y **yacc**, conforme a las restricciones del curso. No se emplean librerías CFG de alto nivel ni `ast` de Python para el parser formal.

1.4. Trazabilidad a la fase léxica

Los IDs de token se preservan en `%token` de `parser.y` (p. ej. `TOKEN_IDENTIFIER = 100`). El archivo `lexico_equipo_11_scanner.l` es la base del scanner integrado; el modo `-t` reproduce la salida del reporte léxico (TC-01–TC-04).

2. Análisis

2.1. Entregables del reto (Sección IV, documento de evaluación)

1. Gramática correcta y no ambigua para kernels Triton, con explicación de decisiones.
2. Gestión de tablas de símbolos durante el análisis sintáctico.
3. Implementación del parser con yacc.
4. Mensajes de error sintácticos.
5. Ejemplos de salida del parser.

2.2. Requerimientos funcionales

- Reconocer programas `.tri`: imports, `@triton.jit`, `def`, cuerpos indentados.
- Sentencias: asignación, `if/elif/else`, `while`, `for/in`, `return`, `pass`.
- Expresiones: literales, operadores (500), llamadas `tl.*`, subíndices `[: , None]`, argumentos con nombre (`mask=`, `dtype=`).
- Rechazar sintaxis inválida con `SYNTAX_ERROR` en español.
- Actualizar `syntab`: imports, funciones, parámetros, variables.

2.3. CFG (resumen)

La gramática formal completa (**33 no terminales**, **91 alternativas** numeradas P-001...P-091, inventario en Tabla 13 del Apéndice A) se documenta en el Listado 11 y el bloque yacc Listado 12. El siguiente extracto resume las construcciones principales para lectura rápida en el análisis.

Listing 1: CFG resumida del lenguaje `.tri` (Triton kernel subset)

```
1 program      -> opt_newlines top_block opt_newlines
2 top_block    -> top_block top_line | epsilon
3 top_line     -> stmt | stmt NEWLINE
4
5 stmt         -> import_stmt | def_stmt | assign_stmt | if_stmt
6              | while_stmt | for_stmt | return_stmt | pass_stmt | expr_stmt
7
8 import_stmt  -> KW_IMPORT import_name opt_as
9 import_name  -> ID | import_name '.' ID
10
11 opt_triton_decorator -> epsilon | '@' ID '.' ID opt_newlines
12
13 def_stmt     -> opt_triton_decorator KW_DEF ID '(' param_list_opt ')' ':' suite
14
15 suite        -> simple_stmt
16              | opt_newlines INDENT block_body DEDENT
17
18 expr         -> literal | ID | ID member_chain
19              | expr OP expr | expr '[' subscript_list ']'
20              | '(' arg_list_opt ')' | call_expr | unary_expr
21
22 subscript    -> expr | NONE | ':' opt_expr | opt_expr ':' opt_expr
23              | opt_expr ':' opt_expr ':' opt_expr
24
25 call_expr    -> ID '(' arg_list_opt ')'
26              | ID member_chain '(' arg_list_opt ')'
27
28 arg          -> expr | ID '=' expr
```

2.4. Pasos para obtener la gramática

1. Inventario de tokens del reporte léxico (Cuadro 1).
2. Identificación de estructuras Triton en 70 kernels de referencia.
3. Borrador de producciones para bloques indentados (`INDENT/DEDENT`).
4. Incorporación de expresiones, llamadas y subíndices tensoriales.
5. Resolución de ambigüedades con precedencias yacc y pruebas iterativas.

2.5. Simplificaciones justificadas

Cuadro 1: Simplificaciones de la CFG.

Tipo	Decisión
Unitarias	Jerarquía de expresiones colapsada en <code>expr</code> con <code>%left</code> .
ϵ	Solo en listas opcionales, <code>else</code> y líneas vacías iniciales.
Símbolos inútiles	Sin <code>class</code> , <code>try</code> , <code>switch</code> (no usados en kernels).
Indentación	<code>WS_COUNT</code> $\rightarrow$ pila $\rightarrow$ <code>INDENT/DEDENT</code> .
Comentarios	Patrón <code>#[\^)\n]*</code> para no consumir <code>)</code> en la misma línea.

2.6. Modificaciones al analizador léxico

- Integración con `y.tab.h`; `yylex()` retorna códigos de token.
- Tokens sintéticos 910/911 (no en Cuadro 1 original; documentados en este reporte).
- Sin nuevos IDs en el Cuadro 1 oficial; trazabilidad numérica conservada.
- **Detección `BAD_ID`** (heredada del analizador léxico del equipo en <https://github.com/G4ndhiV/Analizador-lexico>): identificadores que contienen caracteres prohibidos (`$`, acento grave, barra invertida, `?`, `!`) se reconocen con una regla Flex dedicada *antes* que `ID`, emitiendo `LEXICAL_ERROR` (999) sin fragmentar el lexema.

2.7. Patrones léxicos: `BAD_ID`

Cuadro 2: Caracteres invalidos en identificadores y patron Flex.

Definición	Regex	Comportamiento
<code>BAD_ID_CHAR</code>	<code>[\$'?!]</code>	Conjunto de símbolos no permitidos en nombres.
<code>BAD_ID</code>	<code>ver lexer/lexico_equipo_11_scanner.l</code>	Secuencias con al menos un carácter inválido; prioridad sobre <code>ID</code> .
<code>ID</code>	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	Identificadores válidos (tabla 100).

Mensaje emitido: `LEXICAL_ERROR: ... (identificador con caracteres invalidos)`. El driver marca `lexical_error_seen` y termina con código de salida 2, sin reportar `PARSE_OK`.

2.8. Trazabilidad token léxico $\rightarrow$ yacc

Cuadro 3: Correspondencia Token ID (reporte léxico) y yacc.

ID	Nombre	yacc
100	IDENTIFIER	TOKEN_IDENTIFIER
101	KEYWORD	KW_def, KW_if, ...
200–400	Literales	TOKEN_INTEGER, etc.
500	OPERATOR	TOKEN_OPERATOR
600	DELIMITER	TOKEN_DELIMITER o literales () [] : ,
900	WS_COUNT	convertido a INDENT/DEDENT
950	NEWLINE	TOKEN_NEWLINE
910/911	(sintéticos)	TOKEN_INDENT, TOKEN_DEDENT

3. Diseño

3.1. Arquitectura del sistema

El diseño sigue el modelo en capas del compilador: **entrada** $\rightarrow$ **scanner** $\rightarrow$ **parser** $\rightarrow$ **acciones semánticas** (symtab + salida).

.tri $\rightarrow$ Flex (lexer/) $\rightarrow$ yacc (parser.y) $\rightarrow$ Symtab + stdout/stderr

Figura 1: Pipeline del analizador sintáctico.

3.2. Integración lex $\leftrightarrow$ yacc

1. Bison genera `y.tab.c/h` con códigos de token y tipos `YYSTYPE`.
2. Flex incluye `y.tab.h`; `yflexlex()` reconoce patrones; `yylex()` inyecta `INDENT/DEDENT`.
3. `yylval.i` transporta índices de tabla léxica; `yylval.s` transporta lexemas de operadores y delimitadores.
4. `driver.c` invoca `yyparse()` y, si no hay errores, imprime `PARSE_OK` y tablas.

3.3. Diseño de la especificación yacc

- **Regla inicial:** `program` con bloque superior y newlines opcionales.
- **Bloques:** suite usa `INDENT/DEDENT` para cuerpos de función y ramas `if`.
- **Triton:** `opt_triton_decorator` reconoce `@triton.jit` en líneas separadas del `def`.
- **Expresiones:** `member_chain` para `tl.load`; `subscript_list` para indexación tensorial.
- **Precedencia:** `%left` para operadores; `%nonassoc KW_ELSE` para *dangling else*.

3.4. Tabla de símbolos del parser (diseño)

Cuadro 4: Esquema de entradas en `syntab.c`.

Tipo	Campos	Inserción
Import	módulo, alias, línea	<code>import_stmt</code>
Función	nombre, línea, aridad, <code>triton_jit</code>	cierre de <code>def_stmt</code>
Parámetro	función, nombre, posición, anotación	cada <code>param</code>
Variable	nombre, ámbito, línea	<code>assign_stmt</code> con <code>=</code>

3.5. Mensajes de error (diseño)

- Léxico: `LEXICAL_ERROR: token_id=<999>line=<n>lexeme="..."`.
- Sintáctico: `SYNTAX_ERROR: line=<n>near=" message='...' vía yyerror()`.

3.6. Trazabilidad diseño → análisis

Cada requisito del análisis (Sección 2) se implementa en un módulo: CFG en `parser.y`, indentación en `lexer/`, `syntab` en `syntab.c`, errores en `yyerror`, salidas en `driver.c`.

4. Implementación

4.1. Estructura del proyecto

```
analizador-sintactico/  
  lexer/lexico_equipo_11_scanner.l  
  parser/parser.y, syntab.c, syntab.h  
  src/driver.c  
  build/triton_parser  
  tests/{valid,invalid}/ results/{parser,benchmark}/  
  figures/benchmark_parse_results.pdf
```

4.2. Compilación y ejecución

Instrucciones completas para una PC nueva (clonado, dependencias, salidas): Sección 5.11 y README.md en GitHub.

```
git clone https://github.com/G4ndhiV/Analizador-sint-ctico.git  
cd Analizador-sint-ctico  
make check-deps    # opcional  
make build  
make test          # 12 casos unitarios  
make benchmark     # 70 kernels en benchmark/kernels/  
make figures  
make pdf           # requiere tectonic o latexmk  
./build/triton_parser archivo.tri  
./build/triton_parser -t archivo.tri    # modo solo lexico
```

4.3. Fragmento: detección BAD_ID (lexer)

Misma política que el repositorio léxico del equipo (`Analizador-lexico`): la regla `BAD_ID` tiene precedencia sobre `ID` para no tokenizar por partes un nombre inválido.

Listing 2: Definiciones Flex BAD_ID_CHAR y BAD_ID.

```

1 LETTER      [a-zA-Z_]
2 DIGIT       [0-9]
3 ID          [a-zA-Z_][a-zA-Z0-9_]*
4 /* BAD_ID: misma politica que Analizador-lexico (G4ndhiV/Analizador-lexico) */
5 BAD_ID_CHAR [$'\\"?!
```

Listing 3: Acción léxica: error por identificador con caracteres invalidos.

```

1
2 {BAD_ID} {
3     /* Un solo error lexico por lexema invalido (no se descompone en varios
4        tokens). */
5     fprintf(stderr,
6             "LEXICAL_ERROR: token_id=<%d> line=%d lexeme=\"%s\" (identificador
7             con caracteres invalidos)\n",
8             TOKEN_ERROR, line_no, yytext);
9     lexical_error_seen = 1;
10 }
```

4.4. Fragmento: integración de indentación (lexer)

Listing 4: Pila de indentación y yylex wrapper.

```

1     strncpy(lexeme_buf, text ? text : "", MAX_LEXEME_LEN - 1);
2     lexeme_buf[MAX_LEXEME_LEN - 1] = '\0';
3     yylval.s = lexeme_buf;
4     return lexeme_buf;
5 }
6
7 static int return_token(int tok, const char *lexeme) {
8     save_lexeme(lexeme);
9     return tok;
10 }
11
12 static int push_pending(int tok) {
13     if (pending_count >= (int)(sizeof(pending_tokens) / sizeof(pending_tokens
14         [0]))) {
15         return -1;
16     }
17     pending_tokens[pending_count++] = tok;
18     return 0;
19 }
20
21 static int pop_pending(void) {
22     if (pending_count == 0) {
23         return 0;
24     }
25     return pending_tokens[--pending_count];
26 }
27
28 static int indent_width(const char *text, int len) {
29     int i;
30     int w = 0;
31     for (i = 0; i < len; ++i) {
32         if (text[i] == '_') {
33             w++;
34         } else if (text[i] == '\t') {
35             w += 4;
36         }
37     }
38     return w;
39 }
```

```

39
40 static int handle_indent(int width) {
41     int top = indent_stack[indent_depth - 1];
42     if (width > top) {
43         if (indent_depth >= (int)(sizeof(indent_stack) / sizeof(indent_stack[0])
44             )) {
45             return TOKEN_ERROR;
46         }
47         indent_stack[indent_depth++] = width;
48         return TOKEN_INDENT;
49     }
50     if (width < top) {
51         while (indent_depth > 1 && indent_stack[indent_depth - 1] > width) {
52             indent_depth--;

```

4.5. Fragmento: reglas principales (yacc)

Listing 5: Producciones program, stmt, def_stmt (extracto).

```

1  %nonassoc KW_ELSE
2
3  %start program
4
5  %%
6
7  program
8      : opt_newlines top_block opt_newlines
9        { parse_ok = 1; }
10     ;
11
12  opt_newlines
13      : /* empty */
14        | opt_newlines TOKEN_NEWLINE
15     ;
16
17  top_block
18      : /* empty */
19        | top_block top_line
20     ;
21
22  top_line
23      : stmt
24        | stmt TOKEN_NEWLINE
25     ;
26
27  stmt
28      : import_stmt
29        | def_stmt
30        | assign_stmt
31        | if_stmt
32        | while_stmt
33        | for_stmt
34        | return_stmt
35        | pass_stmt
36        | expr_stmt
37     ;
38
39  import_stmt
40      : KW_IMPORT import_name opt_as
41        {
42            symtab_add_import($2, $3 ? $3 : "", line_no);
43        }

```

```

44     ;
45
46 import_name
47     : TOKEN_IDENTIFIER
48     {
49         $$ = strdup(id_text($1));
50     }
51 | import_name '.' TOKEN_IDENTIFIER
52     {
53         size_t len = strlen($1) + strlen(id_text($3)) + 2;
54         char *buf = (char *)malloc(len);
55         snprintf(buf, len, "%s.%s", $1, id_text($3));
56         free($1);
57         $$ = buf;
58     }
59     ;
60
61 opt_as
62     : /* empty */ { $$ = NULL; }
63 | KW_AS TOKEN_IDENTIFIER { $$ = (char *)id_text($2); }
64     ;
65
66 triton_decorator
67     : '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
68     {
69         if (strcmp(id_text($2), "triton") == 0 && strcmp(id_text($4), "jit")
70             == 0) {
71             triton_decorator_pending = 1;
72         }
73     }
74     ;
75
76 def_stmt
77     : opt_triton_decorator KW_DEF TOKEN_IDENTIFIER

```

4.6. Fragmento: tabla de símbolos

Listing 6: Estructuras e inserción en symtab (extracto).

```

1  #include "symtab.h"
2
3  #include <stdio.h>
4  #include <string.h>
5
6  #define MAX_FUNCS    256
7  #define MAX_PARAMS   1024
8  #define MAX_VARS     1024
9  #define MAX_IMPORTS  64
10
11 static FuncEntry funcs[MAX_FUNCS];
12 static int func_count;
13
14 static ParamEntry params[MAX_PARAMS];
15 static int param_count;
16
17 static VarEntry vars[MAX_VARS];
18 static int var_count;
19
20 static ImportEntry imports[MAX_IMPORTS];
21 static int import_count;
22
23 static char current_scope[64] = "global";

```

```

24
25 static void copy_str(char *dst, size_t n, const char *src) {
26     if (!src) {
27         dst[0] = '\0';
28         return;
29     }
30     strncpy(dst, src, n - 1);
31     dst[n - 1] = '\0';
32 }
33
34 void symtab_reset(void) {
35     func_count = 0;
36     param_count = 0;
37     var_count = 0;
38     import_count = 0;
39     copy_str(current_scope, sizeof(current_scope), "global");
40 }
41
42 void symtab_set_scope(const char *scope) {
43     copy_str(current_scope, sizeof(current_scope), scope ? scope : "global");
44 }
45
46 void symtab_add_import(const char *module, const char *alias, int line) {
47     if (import_count >= MAX_IMPORTS) {
48         return;
49     }
50     copy_str(imports[import_count].module, sizeof(imports[import_count].module),
51             module);
52     copy_str(imports[import_count].alias, sizeof(imports[import_count].alias),
53             alias);
54     imports[import_count].line = line;
55     import_count++;
56 }
57
58 void symtab_add_function(const char *name, int line, int arity, int
59                          is_triton_jit) {
60     if (func_count >= MAX_FUNCS) {
61         return;
62     }
63     copy_str(funcs[func_count].name, sizeof(funcs[func_count].name), name);
64     funcs[func_count].line = line;
65     funcs[func_count].arity = arity;
66     funcs[func_count].is_triton_jit = is_triton_jit;
67     func_count++;
68     symtab_set_scope(name);
69 }
70
71 void symtab_add_param(const char *func, const char *name, int pos, const char *
72                      annot, int line) {
73     if (param_count >= MAX_PARAMS) {
74         return;
75     }
76     copy_str(params[param_count].func, sizeof(params[param_count].func), func);
77     copy_str(params[param_count].name, sizeof(params[param_count].name), name);
78     copy_str(params[param_count].annot, sizeof(params[param_count].annot), annot);
79     params[param_count].position = pos;

```

4.7. Fragmento: manejo de errores

Listing 7: Función yyerror.

```

1      | TOKEN_IDENTIFIER '=' expr
2      ;
3
4  %%
5
6  void yyerror(const char *msg) {
7      fprintf(stderr, "SYNTAX_ERROR: line=%d near=' ' message='%s'\n", line_no, msg
8          );
9  }
10
11 int yyparse_started(void) {
12     parser_mode = 1;
13     lexer_reset_for_parse();
14     syntab_reset();
15     return 0;
16 }

```

4.8. Ejemplo de salida del parser (entregable 5)

Al analizar `triton_kernel_min.tri`:

PARSE_OK: archivo '...' sintacticamente valido.

```

=== PARSER SYMBOL TABLE ===
--- IMPORTS ---
1 line=2 module=triton alias=-
2 line=2 module=triton.language alias=t1
--- FUNCTIONS ---
1 line=5 name=add_kernel arity=5 triton_jit=1
--- PARAMETERS ---
...
=== SYMBOL TABLE: IDENTIFIERS ===
...

```

5. Verificación y Validación

5.1. Resultados experimentales

Hallazgo principal. El analizador sintáctico `triton_parser` alcanza **97.1 % PARSE_OK** sobre el benchmark TRITONHEAL (68/70 kernels) y **100 % de éxito** en la suite unitaria (12/12). Los únicos dos rechazos corresponden a **mutaciones sintácticamente inválidas** del benchmark (`k_0040`, `k_0058`), confirmadas también con el analizador `ast` de Python. **No se registraron falsos rechazos** en kernels válidos ni errores léxicos en la corrida del benchmark.

5.2. Resumen cuantitativo

Cuadro 5: Cuadro resumen de resultados (corridas locales reproducibles con `make test` y `make benchmark`).

Experimento	Éxito	Total	Tasa
Suite unitaria (regresión + inválidos + léxico)	12	12	100 %
Benchmark TRITONHEAL (kernels <code>k_*.tri</code>)	68	70	97.1 %
↪ kernels válidos aceptados	68	68	100 %
↪ mutaciones rechazadas (correcto)	2	2	100 %
Errores léxicos en benchmark	0	70	0 %

5.3. Interpretación de los resultados

Los resultados deben leerse en dos niveles:

1. **Suite unitaria (12/12 PASS).** Valida que el parser cumple los entregables del curso: acepta `ejemplo_1.tri`–`ejemplo_3.tri` del reporte léxico, kernels Triton mínimos propios, rechaza cinco archivos con errores sintácticos deliberados, rechaza un identificador con caracteres invalidos (`BAD_ID`, caso I6) con `LEXICAL_ERROR`, y conserva el modo `-t` del scanner.
2. **Benchmark de 70 kernels (68 PARSE_OK).** Mide desempeño sobre el banco de pruebas TRITONHEAL usado en el proyecto del equipo. De 70 archivos, **68 son sintácticamente válidos** y fueron aceptados; **2 son mutaciones inválidas** sembradas en el benchmark y fueron rechazadas correctamente por el parser.

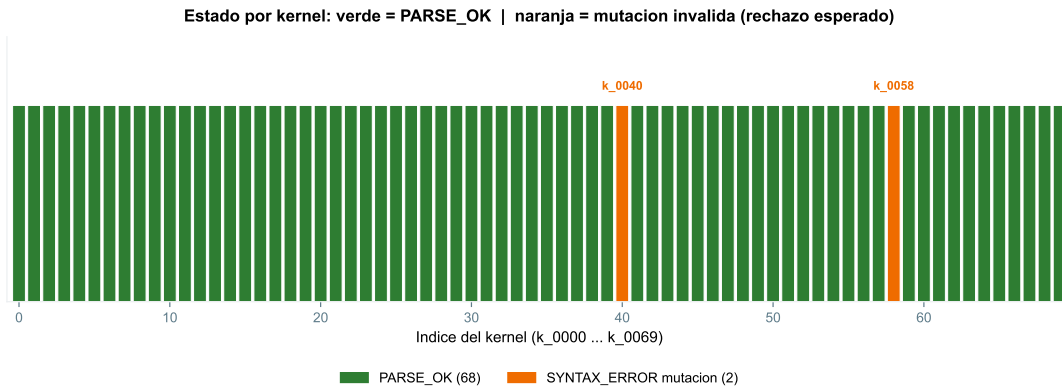
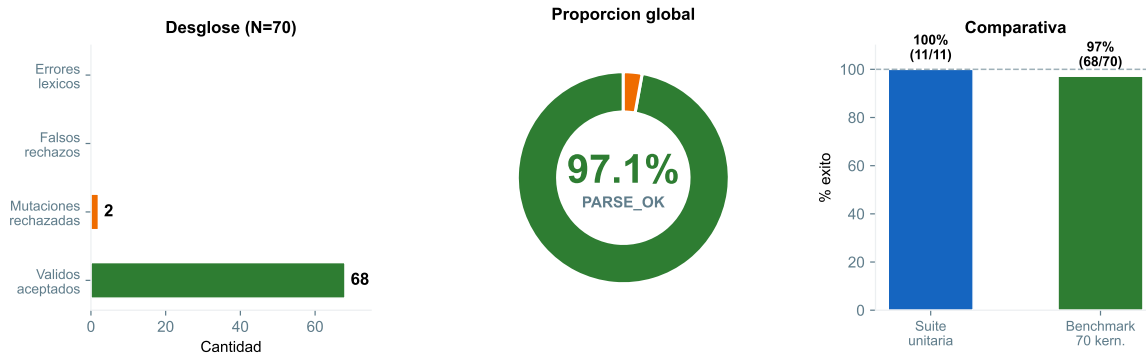
Conclusión: la tasa del 97.1 % *no* indica fallos en código Triton correcto; indica que el 97.1 % del *total del banco* incluye 2 archivos que *deben* fallar. La métrica relevante para código válido es $68/68 = 100\%$ de aceptación.

5.4. Gráfica de resultados (Figura 2)

La Figura 2 presenta de forma explícita: (i) tarjetas con la tasa global y la cobertura real; (ii) desglose por categoría; (iii) diagrama de dona con el 97.1 %; (iv) mapa *kernel por kernel* de los 70 archivos; (v) comparación con la suite unitaria.

Resultados del analizador sintactico — Benchmark TRITONHEAL (70 kernels)

Suite unitaria: 11/11 PASS | Benchmark: 68/70 PARSE_OK (97.1%) | Kernels validos aceptados: 68/68 (100%)



Conclusion: sobre kernels sintacticamente validos, el parser acepta el 100% (68/68). Los 2 SYNTAX_ERROR (k_0040, k_0058) son mutaciones del benchmark; Python ast.parse tambien falla. Suite unitaria: 11/11 PASS. Cero errores lexicos en la corrida.

Figura 2: **Resultados del analizador sintáctico.** Panel superior: indicadores clave. Centro izquierda: desglose (68 válidos aceptados, 2 mutaciones rechazadas, 0 falsos rechazos). Centro derecha: 97.1 % PARSE_OK global. Inferior: mapa de los 70 kernels (k_0000–k_0069); verde = éxito, naranja = mutación inválida (k_0040, k_0058).

5.5. Detalle de los 2 rechazos (comportamiento esperado)

Cuadro 6: Kernels rechazados: mutaciones inválidas del benchmark.

Kernel	Verificación Python	Motivo del rechazo
k_0040.tri	ast.parse falla	tl.store(out_ptr + offs) & ... — paréntesis inválido.
k_0058.tri	ast.parse falla	tl.store(...), mask=...) — coma mal ubicada.

Estos rechazos demuestran que el parser **detecta errores sintácticos** con el mensaje SYNTAX_ERROR, cumpliendo el entregable de mensajes de error del documento de evaluación.

5.6. Modelo de pruebas

Verificación: ¿Se construyó el producto correctamente? — compila, ejecuta, mensajes coherentes.

Validación: ¿Es el producto correcto? — acepta Triton válido y rechaza inválido.

5.7. Suite unitaria (make test)

Cuadro 7: Casos de prueba del equipo (11 ejecuciones).

ID	Descripción	Resultado
V1–V3	ejemplo_1..3.tri (reporte léxico)	PASS
V4–V5	Kernels Triton mínimos	PASS
I1–I5	Sintaxis inválida deliberada	SYNTAX_ERROR
I6	Identificador con BAD_ID (x\$)	LEXICAL_ERROR
L1	Modo -t tokens	PASS
Total PASS		12 / 12

5.8. Evidencia: salida exitosa (ejemplo_1.tri)

Listing 8: Log de éxito (ejemplo_1.tri, extracto).

```

1 PARSE_OK: archivo '/Users/usuario/Desktop/Analizador_sintáctico/analizador-sintactico/
2   tests/fixtures/lexico/ejemplo_1.tri' sintacticamente valido.
3
4 === PARSER SYMBOL TABLE ===
5 --- IMPORTS ---
6 1 line=1 module=triton alias=tl
7 --- FUNCTIONS ---
8 1 line=2 name=kernel arity=2 triton_jit=0
9 --- PARAMETERS ---
10 1 line=2 func=kernel param=chilaquil_power pos=0 annot=-
11 2 line=2 func=kernel param=hambre_level pos=1 annot=-
12 --- VARIABLES ---
13 1 line=6 scope=kernel name=salsa_roja
14 2 line=7 scope=kernel name=cumbia_factor
15
16 === SYMBOL TABLE: IDENTIFIERS ===
17 1 triton
18 2 tl
19 3 kernel
20 4 chilaquil_power
21 5 hambre_level

```

```

21 | 6      salsa_roja
22 | 7      cumbia_factor

```

5.9. Evidencia: error sintáctico (def_sin_dos_puntos.tri)

Listing 9: Rechazo sintáctico esperado (suite unitaria).

```

1 SYNTAX_ERROR: line=2 near=' ' message='syntax_error, unexpected_TOKEN_NEWLINE, expecting_
  : '

```

5.10. Evidencia: error léxico BAD_ID (I6)

Listing 10: Rechazo por identificador con \$ (regla BAD_ID, alineada con Analizador-lexico).

```

1 LEXICAL_ERROR: token_id=<999> line=2 lexeme="x$" (identificador con caracteres invalidos)

```

5.11. Reproducibilidad

Todo el material para reproducir el proyecto en una **PC nueva** está en el repositorio público (sin dependencias externas de otros repos):

- Repositorio: <https://github.com/G4ndhiV/Analizador-sint-ctico>
- Kernels de prueba: benchmark/kernels/ (70 archivos k_*.tri)
- Fixtures léxicos: tests/fixtures/lexico/
- CFG documentada: report/cfg/triton_cfg.bnf
- Resultados de referencia: results/parser/, results/benchmark/
- Informe PDF: deliverables/Informe_Analizador_Sintactico_Equipo11.pdf

5.11.1. Prerrequisitos

Cuadro 8: Software necesario para reproducir compilación, pruebas, gráfica e informe.

Herramienta	Obligatorio	Instalación típica
git	Sí	Clonar el repositorio
cc (GCC/Clang)	Sí	Xcode CLT (macOS) o build-essential (Linux)
Bison	Sí	brew install bison / apt install bison
Flex	Sí	brew install flex / apt install flex
Python 3 + venv	Sí (gráfica)	python3 -m venv .venv; pip install -r requirements.txt
Tectonic o latexmk	Solo para make pdf	brew install tectonic

Verificación rápida: `make check-deps` (objetivo del Makefile raíz).

5.11.2. Pasos desde cero (PC nueva)

1. Clonar:

```
git clone https://github.com/G4ndhiV/Analizador-sint-ctico.git
cd Analizador-sint-ctico
```

2. Comprobar dependencias (opcional): `make check-deps`

3. Compilar: `make build` → ejecutable `build/triton_parser`

4. Suite unitaria: `make test` → **12/12 PASS**; logs en `results/parser/`

5. Benchmark 70 kernels: `make benchmark` → `results/benchmark/summary.json`

6. Gráfica: `make figures` → `figures/benchmark_parse_results.pdf`

7. Informe PDF: `make pdf` → ver ruta en `deliverables/Informe_Analizador_Sintactico_Equipo11.pdf`

5.11.3. Comando único (regeneración completa)

```
make clean build test benchmark figures pdf
```

5.11.4. Salidas verificables

- `results/parser/summary.json`: "pass": 12, "total": 12.
- `results/benchmark/summary.json`: parse_ok: 68, total: 70.
- `figures/benchmark_parse_results.pdf` y `figures/benchmark_parse_results.png`.
- PDF del informe en `deliverables/` (Apéndice A: CFG de 91 producciones).

5.11.5. Consulta sin recompilar

Basta con clonar el repositorio y abrir el PDF en `deliverables/` y los archivos `results/parser/summary.json` y `results/benchmark/summary.json`.

5.11.6. Notas por sistema operativo

- **macOS**: instalar `bison` y `flex` con Homebrew; el `Makefile` enlaza la librería Flex de Homebrew.
- **Linux**: usar `bison` y `flex` del sistema; enlace con `-lfl`.
- **BAD_ID**: reglas en `lexer/lexico_equipo_11_scanner.l` (repo léxico: <https://github.com/G4ndhiV/Analizador-lexico>); prueba I6 en `tests/invalid/identificador_caracter_invalido.tri`.

Guía extendida: `README.md` en la raíz del repositorio.

6. Plan de Trabajo

Cuadro 9: Cronograma de la fase sintáctica (Equipo 11).

Fase	Actividad	Entregable
1	Revisión tokens léxicos y kernels TRITONHEAL	Inventario de requisitos
2	Diseño CFG + resolución indentación	Documento de análisis
3	Especificación yacc + symtab	<code>parser.y</code> , <code>syntab.c</code>
4	Integración Flex-yacc + driver	<code>triton_parser</code>
5	Suite de pruebas + benchmark 70	<code>tests/</code> , <code>results/</code>
6	Reporte IEEE-830 + gráfica	PDF final

6.1. Distribución de responsabilidades

- Gramática y yacc: análisis conjunto del equipo, implementación en `parser.y`.
- Lexer integrado: extensión de `lexico_equipo_11_scanner.l`.
- Pruebas y benchmark: scripts `run_tests.sh`, `run_benchmark_kernels.sh`.
- Reporte y figuras: $\text{\LaTeX}$ + `plot_benchmark_results.py`.

A. Gramática libre de contexto completa

Esta sección es la **referencia formal exhaustiva** del analizador: cada no terminal, cada alternativa de producción y cada terminal declarado en `parser/parser.y` quedan documentados sin omisiones. El listado numerado P-001...P-091 en el Archivo 11 corresponde **uno a uno** con las 91 alternativas de las 33 reglas yacc (líneas 76–321).

A.1. Convenciones y notación

- **Forma:** `NT ->cuerpo`; varias líneas con el mismo NT son alternativas (OR).
- ε / **epsilon**: producción vacía (`/* empty */` en Bison).
- **Prefijo P-NNN**: identificador de trazabilidad en `triton_cfg.bnf`; no es un símbolo del lenguaje.
- **Terminales léxicos**: se escriben como en yacc (`TOKEN_*`, `KW_*`) o como literales entre comillas simples.
- **Acciones semánticas**: no forman parte de la CFG; se listan en la Subsección A.6 solo como notas de implementación.

A.2. Símbolo inicial y conteo

Cuadro 10: Resumen formal de la gramática implementada.

Concepto	Valor
Símbolo inicial (<code>%start</code>)	<code>program</code>
No terminales con producciones	33
Alternativas de producción (total)	91
Archivo de implementación	<code>parser/parser.y</code>
Archivo BNF documentado	<code>report/cfg/triton_cfg.bnf</code>

A.3. Terminales del analizador

A.3.1. Terminales que aparecen en producciones

Cuadro 11: Terminales usados en al menos una producción de la CFG.

Símbolo yacc	ID	Rol en kernels <code>.tri</code>
<code>TOKEN_IDENTIFIER</code>	100	Nombres, módulos, funciones
<code>TOKEN_INTEGER</code>	200	Literales enteros
<code>TOKEN_FLOAT</code>	300	Literales flotantes
<code>TOKEN_STRING</code>	400	Cadenas
<code>TOKEN_OPERATOR</code>	500	Operadores binarios/unarios, <code>=</code> en asignación
<code>TOKEN_DELIMITER</code>	600	Delimitadores en expresiones y <code>member_chain</code>
<code>TOKEN_NEWLINE</code>	950	Fin de línea
<code>TOKEN_INDENT</code>	910	Inicio de bloque (desde <code>WS_COUNT</code>)
<code>TOKEN_DEDENT</code>	911	Fin de bloque
<code>KW_IMPORT, KW_AS</code>	101	<code>import/as</code>
<code>KW_DEF</code>	101	Definición de kernel
<code>KW_IF, KW_ELIF, KW_ELSE</code>	101	Condicionales
<code>KW_WHILE, KW_FOR, KW_IN</code>	101	Bucles
<code>KW_RETURN, KW_PASS</code>	101	Control de flujo
<code>KW_NONE, KW_TRUE, KW_FALSE</code>	101	Constantes
<code>'@', '.', '(', ')'</code>	—	Decorador, acceso, agrupación
<code>'[', ']', ':', ',', ''</code>	—	Subíndices, slices, listas

A.3.2. Terminales declarados y no usados en reglas

En `parser.y` (líneas 53–59) se declaran además `TOKEN_COMMENT`, `TOKEN_KEYWORD` y `KW_BREAK`, `KW_CONTINUE`, `KW_AND`, `KW_OR`, `KW_NOT`. **Ninguna producción** de la CFG los referencia: el lexer no los entrega al parser en modo normal (`COMMENT` se descarta; las palabras no modeladas en `stmt/expr` producirían error sintáctico si se usaran).

A.4. Precedencia y asociatividad (yacc)

Cuadro 12: Directivas `%left` / `%nonassoc` en `parser.y` (L64–70).

Nivel	Asoc.	Terminales
1 (más baja)	izquierda	TOKEN_NEWLINE
2	izquierda	KW_OR
3	izquierda	KW_AND
4	no asoc.	KW_NOT
5	izquierda	TOKEN_OPERATOR, TOKEN_DELIMITER (en <code>expr</code>)
6	no asoc.	KW_ELIF
7 (más alta)	no asoc.	KW_ELSE

Nota: la jerarquía de `expr` está **colapsada** en un solo no terminal; la precedencia entre operadores concretos (+, *, etc.) depende del orden de lectura y de estas declaraciones, no de niveles `expr_add`, `expr_mul`, etc.

A.5. Inventario de no terminales (33)

Cuadro 13: Catálogo de no terminales: número de alternativas e intervalo en `parser.y`.

No terminal	Alt.	IDs P-	yacc
<code>program</code>	1	001	L76–79
<code>opt_newlines</code>	2	002–003	L81–84
<code>top_block</code>	2	004–005	L86–89
<code>top_line</code>	2	006–007	L91–94
<code>suite</code>	2	008–009	L189–192
<code>block_body</code>	2	010–011	L194–197
<code>block_stmt</code>	3	012–014	L199–203
<code>simple_stmt</code>	1	015	L205–207
<code>stmt</code>	9	016–024	L96–106
<code>import_stmt</code>	1	025	L108–113
<code>import_name</code>	2	026–027	L115–128
<code>opt_as</code>	2	028–029	L130–133
<code>triton_decorator</code>	1	030	L135–142
<code>opt_triton_decorator</code>	2	031–032	L159–162
<code>def_stmt</code>	1	033	L144–157
<code>assign_stmt</code>	1	034	L209–216
<code>if_stmt</code>	1	035	L218–220
<code>elif_parts</code>	2	036–037	L222–225
<code>else_part</code>	2	038–039	L227–230
<code>while_stmt</code>	1	040	L232–234
<code>for_stmt</code>	1	041	L236–238
<code>return_stmt</code>	2	042–043	L240–243
<code>pass_stmt</code>	1	044	L245–247
<code>expr_stmt</code>	1	045	L249–251
<code>param_list_opt</code>	2	046–047	L164–167
<code>param_list</code>	2	048–049	L169–174
<code>param</code>	2	050–051	L176–187

(continuación)

No terminal	Alt.	IDs P-	yacc
expr	16	052–068	L253–271
opt_expr	2	069–070	L273–276
unary_expr	1	071	L291–293
member_chain	4	072–075	L295–300
call_expr	3	076–078	L302–306
subscript_list	2	079–080	L278–281
subscript	5	081–085	L283–289
arg_list_opt	2	086–087	L308–311
arg_list	2	088–089	L313–316
arg	2	090–091	L318–321
Total alternativas:		91	

A.6. Notas semánticas ligadas a la CFG (fuera de producciones)

Cuadro 14: Acciones en C asociadas a reglas seleccionadas (no son parte de la CFG).

Regla	Efecto
program	parse_ok := 1 al reducir.
import_stmt	syntab_add_import.
triton_decorator	Activa bandera si lexemas son triton.jit.
def_stmt	Tras ID: current_func; tras ')': syntab_add_function.
param	syntab_add_param por cada parámetro.
assign_stmt	Variable en syntab solo si TOKEN_OPERATOR es '='.

A.7. Desglose detallado: no terminal expr (16 alternativas)

Todas las formas de expresión admitidas por el parser:

1. TOKEN_INTEGER
2. TOKEN_FLOAT
3. TOKEN_STRING
4. TOKEN_IDENTIFIER (solo identificador)
5. TOKEN_IDENTIFIER member_chain (identificador calificado)
6. KW_NONE, KW_TRUE, KW_FALSE
7. expr TOKEN_OPERATOR expr (binario)
8. expr TOKEN_DELIMITER expr (binario con delimitador léxico)
9. '(' arg_list_opt ')' (tupla / agrupación)
10. '[' expr ']' (lista de un elemento)
11. '[' expr ':' expr ']' (slice con dos extremos)
12. '[' expr ':' expr ',' expr ']' (slice con stride)
13. expr '[' subscript_list ']' (indexación)
14. call_expr
15. unary_expr

A.8. Desglose detallado: subscript (5 alternativas)

Soporta índices y slices estilo Python/Triton: expresión simple, None, : *expr*, *expr* : *expr*, *expr* : *expr* : *expr* (con *opt_expr* opcional en cada posición de slice).

A.9. Desglose detallado: block_stmt (3 alternativas)

Permite sentencias sueltas, líneas vacías dentro de bloques indentados, y secuencias NEWLINE *stmt* para alinear el cuerpo con la indentación Python.

A.10. Listado completo de producciones (P-001 a P-091)

Listing 11: CFG completa numerada: 33 no terminales, 91 alternativas (report/cfg/triton_cfg.bnf).

```
1  % =====
2  % CFG COMPLETA - Analizador sintactico Triton (.tri) - Equipo 11
3  % Fuente canonica de implementacion: parser/parser.y (reglas entre %% ... %)
4  % Simbolo inicial: program
5  % No terminales: 33 | Producciones (alternativas): 91 | Ultima revision: parser.y
6  % Notacion: NT -> cuerpo      (una alternativa por línea con prefijo P-NNN)
7  % epsilon = produccion vacia (/* empty */ en yacc)
8  % =====
9
10 % -----
11 % 1. PROGRAMA Y ESTRUCTURA DE BLOQUES
12 % -----
13
14 % NT: program (1 produccion) | parser.y L76-79
15 P-001  program          -> opt_newlines top_block opt_newlines
16
17 % NT: opt_newlines (2) | L81-84
18 P-002  opt_newlines     -> epsilon
19 P-003  opt_newlines     -> opt_newlines TOKEN_NEWLINE
20
21 % NT: top_block (2) | L86-89
22 P-004  top_block        -> epsilon
23 P-005  top_block        -> top_block top_line
24
25 % NT: top_line (2) | L91-94
26 P-006  top_line         -> stmt
27 P-007  top_line         -> stmt TOKEN_NEWLINE
28
29 % NT: suite (2) | L189-192
30 P-008  suite             -> simple_stmt
31 P-009  suite             -> opt_newlines TOKEN_INDENT block_body TOKEN_DEDENT
32
33 % NT: block_body (2) | L194-197
34 P-010  block_body        -> epsilon
35 P-011  block_body        -> block_body block_stmt
36
37 % NT: block_stmt (3) | L199-203
38 P-012  block_stmt        -> stmt
39 P-013  block_stmt        -> block_body TOKEN_NEWLINE
40 P-014  block_stmt        -> block_body TOKEN_NEWLINE stmt
41
42 % NT: simple_stmt (1) | L205-207
43 P-015  simple_stmt       -> stmt
44
45 % -----
46 % 2. SENTENCIAS (stmt y derivados)
47 % -----
48
49 % NT: stmt (9 alternativas) | L96-106
50 P-016  stmt              -> import_stmt
51 P-017  stmt              -> def_stmt
52 P-018  stmt              -> assign_stmt
53 P-019  stmt              -> if_stmt
54 P-020  stmt              -> while_stmt
```

```

55 P-021 stmt -> for_stmt
56 P-022 stmt -> return_stmt
57 P-023 stmt -> pass_stmt
58 P-024 stmt -> expr_stmt
59
60 % NT: import_stmt (1) / L108-113
61 P-025 import_stmt -> KW_IMPORT import_name opt_as
62
63 % NT: import_name (2) / L115-128
64 P-026 import_name -> TOKEN_IDENTIFIER
65 P-027 import_name -> import_name '.' TOKEN_IDENTIFIER
66
67 % NT: opt_as (2) / L130-133
68 P-028 opt_as -> epsilon
69 P-029 opt_as -> KW_AS TOKEN_IDENTIFIER
70
71 % NT: triton_decorator (1) / L135-142
72 % Semantica: si lex(ID1)=triton y lex(ID2)=jit entonces triton_decorator_pending:=1
73 P-030 triton_decorator -> '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
74
75 % NT: opt_triton_decorator (2) / L159-162
76 P-031 opt_triton_decorator -> epsilon
77 P-032 opt_triton_decorator -> triton_decorator opt_newlines
78
79 % NT: def_stmt (1 cadena, 3 puntos de accion en yacc) / L144-157
80 % Forma BNF equivalente a la secuencia yacc:
81 % [opt_triton_decorator] KW_DEF ID {A} '(' param_list_opt ')' ':' {B} suite
82 % A: current_func := lex(ID); param_pos_counter := 0
83 % B: symtab_add_function(current_func, arity=param_list_opt, triton_jit=flag)
84 P-033 def_stmt -> opt_triton_decorator KW_DEF TOKEN_IDENTIFIER '('
      param_list_opt ')' ':' suite
85
86 % NT: assign_stmt (1) / L209-216
87 % Semantica: solo si lex(TOKEN_OPERATOR)=='=' se registra variable en symtab
88 P-034 assign_stmt -> TOKEN_IDENTIFIER TOKEN_OPERATOR expr
89
90 % NT: if_stmt (1) / L218-220
91 P-035 if_stmt -> KW_IF expr ':' suite elif_parts else_part
92
93 % NT: elif_parts (2, recursivo a izquierda) / L222-225
94 P-036 elif_parts -> epsilon
95 P-037 elif_parts -> elif_parts KW_ELIF expr ':' suite
96
97 % NT: else_part (2) / L227-230
98 P-038 else_part -> epsilon
99 P-039 else_part -> KW_ELSE ':' suite
100
101 % NT: while_stmt (1) / L232-234
102 P-040 while_stmt -> KW_WHILE expr ':' suite
103
104 % NT: for_stmt (1) / L236-238
105 P-041 for_stmt -> KW_FOR TOKEN_IDENTIFIER KW_IN expr ':' suite
106
107 % NT: return_stmt (2) / L240-243
108 P-042 return_stmt -> KW_RETURN expr
109 P-043 return_stmt -> KW_RETURN
110
111 % NT: pass_stmt (1) / L245-247
112 P-044 pass_stmt -> KW_PASS
113
114 % NT: expr_stmt (1) / L249-251
115 P-045 expr_stmt -> expr
116
117 % -----
118 % 3. PARAMETROS DE FUNCION
119 % -----
120
121 % NT: param_list_opt (2) / L164-167
122 P-046 param_list_opt -> epsilon
123 P-047 param_list_opt -> param_list
124
125 % NT: param_list (2, lista separada por coma, recursiva) / L169-174
126 P-048 param_list -> param

```

```

127 P-049 param_list      -> param_list ',' param
128
129 % NT: param (2) / L176-187
130 P-050 param          -> TOKEN_IDENTIFIER
131 P-051 param          -> TOKEN_IDENTIFIER ':' expr
132
133 % -----
134 % 4. EXPRESIONES, LLAMADAS, SUBINDICES Y ARGUMENTOS
135 % -----
136
137 % NT: expr (16 alternativas) / L253-271
138 % Precedencia resuelta por %left/%nonassoc en parser.y L64-70 (ver apéndice)
139 P-052 expr           -> TOKEN_INTEGER
140 P-053 expr           -> TOKEN_FLOAT
141 P-054 expr           -> TOKEN_STRING
142 P-055 expr           -> TOKEN_IDENTIFIER
143 P-056 expr           -> TOKEN_IDENTIFIER member_chain
144 P-057 expr           -> KW_NONE
145 P-058 expr           -> KW_TRUE
146 P-059 expr           -> KW_FALSE
147 P-060 expr           -> expr TOKEN_OPERATOR expr
148 P-061 expr           -> expr TOKEN_DELIMITER expr
149 P-062 expr           -> '(' arg_list_opt ') '
150 P-063 expr           -> '[' expr ']'
151 P-064 expr           -> '[' expr ':' expr ']'
152 P-065 expr           -> '[' expr ':' expr ',' expr ']'
153 P-066 expr           -> expr '[' subscript_list ']'
154 P-067 expr           -> call_expr
155 P-068 expr           -> unary_expr
156
157 % NT: opt_expr (2) / L273-276
158 P-069 opt_expr       -> epsilon
159 P-070 opt_expr       -> expr
160
161 % NT: unary_expr (1) / L291-293
162 P-071 unary_expr     -> TOKEN_OPERATOR expr
163
164 % NT: member_chain (4) / L295-300
165 % Cubre atributos .name y formas con TOKEN_DELIMITER (p.ej. acceso tl.*)
166 P-072 member_chain   -> '.' TOKEN_IDENTIFIER
167 P-073 member_chain   -> member_chain '.' TOKEN_IDENTIFIER
168 P-074 member_chain   -> TOKEN_DELIMITER TOKEN_IDENTIFIER
169 P-075 member_chain   -> member_chain TOKEN_DELIMITER TOKEN_IDENTIFIER
170
171 % NT: call_expr (3) / L302-306
172 P-076 call_expr      -> TOKEN_IDENTIFIER '(' arg_list_opt ') '
173 P-077 call_expr      -> TOKEN_IDENTIFIER member_chain '(' arg_list_opt ') '
174 P-078 call_expr      -> TOKEN_IDENTIFIER '(' ') '
175
176 % NT: subscript_list (2) / L278-281
177 P-079 subscript_list -> subscript
178 P-080 subscript_list -> subscript_list ',' subscript
179
180 % NT: subscript (5) / L283-289
181 % Incluye slices : , : , : con opt_expr opcional
182 P-081 subscript      -> expr
183 P-082 subscript      -> KW_NONE
184 P-083 subscript      -> ':' opt_expr
185 P-084 subscript      -> opt_expr ':' opt_expr
186 P-085 subscript      -> opt_expr ':' opt_expr ':' opt_expr
187
188 % NT: arg_list_opt (2) / L308-311
189 P-086 arg_list_opt    -> epsilon
190 P-087 arg_list_opt    -> arg_list
191
192 % NT: arg_list (2) / L313-316
193 P-088 arg_list        -> arg
194 P-089 arg_list        -> arg_list ',' arg
195
196 % NT: arg (2) / L318-321
197 P-090 arg             -> expr
198 P-091 arg             -> TOKEN_IDENTIFIER '=' expr
199

```

```

200 % =====
201 % TERMINALES EN parser.y (%token) - INVENTARIO COMPLETO
202 % =====
203 % Usados directamente en producciones de la CFG anterior:
204 % TOKEN_IDENTIFIER, TOKEN_INTEGER, TOKEN_FLOAT, TOKEN_STRING,
205 % TOKEN_OPERATOR, TOKEN_DELIMITER, TOKEN_NEWLINE, TOKEN_INDENT, TOKEN_DEDENT,
206 % KW_DEF, KW_RETURN, KW_IF, KW_ELIF, KW_ELSE, KW_WHILE, KW_FOR, KW_IN,
207 % KW_IMPORT, KW_AS, KW_PASS, KW_NONE, KW_TRUE, KW_FALSE,
208 % Literales: '@' '.' '(' ')' '[' ']' ':' ',' ,
209 %
210 % Declarados en parser.y pero SIN regla gramatical que los consuma (reserva):
211 % TOKEN_COMMENT (700) - descartado en lexer antes del parser
212 % TOKEN_KEYWORD (101) - canal generico; el lexer emite KW_* especificos
213 % KW_BREAK, KW_CONTINUE, KW_AND, KW_OR, KW_NOT
214 % (si aparecen en fuente, el lexer puede emitirlos; no hay produccion stmt/expr)
215 %
216 % Errores (fuera de CFG, manejo del driver/lexer):
217 % TOKEN_ERROR (999) - BAD_ID, cadena sin cerrar, simbolo suelto '.'
218 %
219 % =====
220 % ALIAS USADOS EN DOCUMENTACION RESUMIDA (Seccion Analisis)
221 % =====
222 % ID=TOKEN_IDENTIFIER INT=TOKEN_INTEGER FLOAT=TOKEN_FLOAT STRING=TOKEN_STRING
223 % OP=TOKEN_OPERATOR DELIM=TOKEN_DELIMITER
224 % NEWLINE=TOKEN_NEWLINE INDENT=TOKEN_INDENT DEDENT=TOKEN_DEDENT

```

A.11. Implementación yacc (referencia cruzada)

Para verificación línea a línea, las reglas ejecutables (con acciones) son:

Listing 12: parser/parser.y: bloque completo de producciones yacc (L76–321).

```

1 program
2   : opt_newlines top_block opt_newlines
3     { parse_ok = 1; }
4   ;
5
6 opt_newlines
7   : /* empty */
8     | opt_newlines TOKEN_NEWLINE
9   ;
10
11 top_block
12   : /* empty */
13     | top_block top_line
14   ;
15
16 top_line
17   : stmt
18     | stmt TOKEN_NEWLINE
19   ;
20
21 stmt
22   : import_stmt
23     | def_stmt
24     | assign_stmt
25     | if_stmt
26     | while_stmt
27     | for_stmt
28     | return_stmt
29     | pass_stmt
30     | expr_stmt
31   ;
32
33 import_stmt
34   : KW_IMPORT import_name opt_as

```

```

35     {
36         symtab_add_import($2, $3 ? $3 : "", line_no);
37     }
38 ;
39
40 import_name
41 : TOKEN_IDENTIFIER
42 {
43     $$ = strdup(id_text($1));
44 }
45 | import_name '.' TOKEN_IDENTIFIER
46 {
47     size_t len = strlen($1) + strlen(id_text($3)) + 2;
48     char *buf = (char *)malloc(len);
49     snprintf(buf, len, "%s.%s", $1, id_text($3));
50     free($1);
51     $$ = buf;
52 }
53 ;
54
55 opt_as
56 : /* empty */ { $$ = NULL; }
57 | KW_AS TOKEN_IDENTIFIER { $$ = (char *)id_text($2); }
58 ;
59
60 triton_decorator
61 : '@' TOKEN_IDENTIFIER '.' TOKEN_IDENTIFIER
62 {
63     if (strcmp(id_text($2), "triton") == 0 && strcmp(id_text($4), "jit")
64         == 0) {
65         triton_decorator_pending = 1;
66     }
67 }
68 ;
69
70 def_stmt
71 : opt_triton_decorator KW_DEF TOKEN_IDENTIFIER
72 {
73     strncpy(current_func, id_text($3), sizeof(current_func) - 1);
74     current_func[sizeof(current_func) - 1] = '\0';
75     param_pos_counter = 0;
76 }
77 | '(' param_list_opt ')' ':'
78 {
79     symtab_add_function(current_func, line_no, $6,
80         triton_decorator_pending);
81     triton_decorator_pending = 0;
82 }
83 suite
84 ;
85
86 opt_triton_decorator
87 : /* empty */
88 | triton_decorator opt_newlines
89 ;
90
91 param_list_opt
92 : /* empty */ { $$ = 0; }
93 | param_list { $$ = $1; }
94 ;
95
96 param_list
97 : param

```

```

96     { $$ = 1; }
97 | param_list ',' param
98 { $$ = $1 + 1; }
99 ;
100
101 param
102 : TOKEN_IDENTIFIER
103 {
104     symtab_add_param(current_func, id_text($1), param_pos_counter++, "",
105                     line_no);
106     $$ = 1;
107 }
108 | TOKEN_IDENTIFIER ':' expr
109 {
110     symtab_add_param(current_func, id_text($1), param_pos_counter++, "",
111                     line_no);
112     $$ = 1;
113 }
114 ;
115
116 suite
117 : simple_stmt
118 | opt_newlines TOKEN_INDENT block_body TOKEN_DEDENT
119 ;
120
121 block_body
122 : /* empty */
123 | block_body block_stmt
124 ;
125
126 block_stmt
127 : stmt
128 | block_body TOKEN_NEWLINE
129 | block_body TOKEN_NEWLINE stmt
130 ;
131
132 simple_stmt
133 : stmt
134 ;
135
136 assign_stmt
137 : TOKEN_IDENTIFIER TOKEN_OPERATOR expr
138 {
139     if (delim_is($2, "=")) {
140         symtab_add_variable(id_text($1), line_no, current_func[0] ?
141                             current_func: "global");
142     }
143 }
144 ;
145
146 if_stmt
147 : KW_IF expr ':' suite elif_parts else_part
148 ;
149
150 elif_parts
151 : /* empty */
152 | elif_parts KW_ELIF expr ':' suite
153 ;
154
155 else_part
156 : /* empty */
157 | KW_ELSE ':' suite
158 ;

```

```

156
157 while_stmt
158     : KW_WHILE expr ':' suite
159     ;
160
161 for_stmt
162     : KW_FOR TOKEN_IDENTIFIER KW_IN expr ':' suite
163     ;
164
165 return_stmt
166     : KW_RETURN expr
167     | KW_RETURN
168     ;
169
170 pass_stmt
171     : KW_PASS
172     ;
173
174 expr_stmt
175     : expr
176     ;
177
178 expr
179     : TOKEN_INTEGER { $$ = $1; }
180     | TOKEN_FLOAT { $$ = $1; }
181     | TOKEN_STRING { $$ = $1; }
182     | TOKEN_IDENTIFIER { $$ = $1; }
183     | TOKEN_IDENTIFIER member_chain { $$ = $1; }
184     | KW_NONE { $$ = 0; }
185     | KW_TRUE { $$ = 0; }
186     | KW_FALSE { $$ = 0; }
187     | expr TOKEN_OPERATOR expr { $$ = 0; }
188     | expr TOKEN_DELIMITER expr { $$ = 0; }
189     | '(' arg_list_opt ')' { $$ = 0; }
190     | '[' expr ']' { $$ = $2; }
191     | '[' expr ':' expr ']' { $$ = 0; }
192     | '[' expr ':' expr ',' expr ']' { $$ = 0; }
193     | expr '[' subscript_list ']' { $$ = 0; }
194     | call_expr { $$ = 0; }
195     | unary_expr { $$ = $1; }
196     ;
197
198 opt_expr
199     : /* empty */ { $$ = 0; }
200     | expr { $$ = $1; }
201     ;
202
203 subscript_list
204     : subscript
205     | subscript_list ',' subscript
206     ;
207
208 subscript
209     : expr { $$ = $1; }
210     | KW_NONE { $$ = 0; }
211     | ':' opt_expr { $$ = 0; }
212     | opt_expr ':' opt_expr { $$ = 0; }
213     | opt_expr ':' opt_expr ':' opt_expr { $$ = 0; }
214     ;
215
216 unary_expr
217     : TOKEN_OPERATOR expr { $$ = $2; }
218     ;

```

```

219
220 member_chain
221     : '.' TOKEN_IDENTIFIER
222     | member_chain '.' TOKEN_IDENTIFIER
223     | TOKEN_DELIMITER TOKEN_IDENTIFIER
224     | member_chain TOKEN_DELIMITER TOKEN_IDENTIFIER
225     ;
226
227 call_expr
228     : TOKEN_IDENTIFIER '(' arg_list_opt ')' { $$ = 0; }
229     | TOKEN_IDENTIFIER member_chain '(' arg_list_opt ')' { $$ = 0; }
230     | TOKEN_IDENTIFIER '(' ')' { $$ = 0; }
231     ;
232
233 arg_list_opt
234     : /* empty */
235     | arg_list
236     ;
237
238 arg_list
239     : arg
240     | arg_list ',' arg
241     ;
242
243 arg
244     : expr
245     | TOKEN_IDENTIFIER '=' expr
246     ;

```

A.12. Trazabilidad y mantenimiento

Cualquier modificación en `parser/parser.y` exige actualizar `triton_cfg.bnf` (renumerar P-*, tabla 13 y conteo 91) y recompilar el PDF. La Sección 2 del análisis conserva un **resumen**; este apéndice es la definición **completa**.

B. Referencias

1. R. J. Castelló, “Chapter 3: Syntax Analysis, Part I — Grammar Processing,” *Module #3: Compilers* (TC3002B, Advanced Applications Development), notas de curso, ITESM, s. f.
2. R. J. Castelló, “Chapter 2: Lexical Analysis,” *Module #3: Compilers* (TC3002B), notas de curso, ITESM, s. f.
3. Equipo 11, *Analizador Léxico — Triton GPU Kernel*, reporte TC3002B, entrega mayo 2026; implementación Flex en <https://github.com/G4ndhiV/Analizador-lexico> (lexico_equipo_11_scanner.1, reglas BAD_ID).
4. Salvador Hinojosa, *Evaluation Metrics Syntax Analyzer GDL Triton*, TC3002B, documento de evaluación, 2026.
5. OpenAI, *Triton: Open-Source GPU Programming*, [en línea]; disponible en <https://triton-lang.org>; Internet.
6. A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Addison-Wesley, 2007.
7. Proyecto TRITONHEAL, benchmark de 70 kernels `k_*.tri`, repositorio de referencia del Equipo 11, 2026.